

Document number	P3266R0
Date	2024-05-05
Reply-to	Jarrad J. Waterloo <descender76 at gmail dot com>
Audience	Evolution Working Group (EWG)

non referenceable types

Table of contents

- non referenceable types
 - Abstract
 - Motivational Examples
 - Motivation
 - More Potential Examples

Abstract

Consumers of pure reference types with shallow const semantics would benefit by disallowing taking a reference to the type.

Motivational Examples

given

```
template<class... S> class function_ref;    // not defined

template<class R, class... ArgTypes>
class function_ref<R(ArgTypes...) cv noexcept(noex)> referenceable(false) // disengage
{
    // ...
};

template <class T>
class optional<T> referenceable(false) // disengage being able to reference i.e.
{
    // ...
};
```

usage

```
// don't allow taking a reference to a non referenceable type
// as they are meant to be only copied
function_ref<void ()>& f(function_ref<void ()>&); // ill formed
optional<int&>& f(optional<int&>&); // ill formed

void action(){}

int main()
{
    function_ref<void ()> fr1{ action };
    // don't allow taking a reference to a non referenceable type
    // as they are meant to be only copied
    function_ref<void ()>& fr2 = fr1; // ill formed

    int i = 42;
    optional<int&> oi1{ i }; // const by default
    // don't allow taking a reference to a non referenceable type
    // as they are meant to be only copied
    optional<int&>& oi2 = oi1; // ill formed

    return 0;
}
```

Motivation

Currently a programmer can not create a reference to a reference as references only support a single level of indirection. Further, taking the address of a reference does not refer to the address of the reference but rather what the referent that the reference refers to. Allowing programmers to create non referenceable types would mean allowing programmers to create reference types that are more consistent with references. This would remove incorrect usage of reference types and prevent that creation of references which are largely useless and only serve to provide additional avenues to dangling.

More Potential Examples

```
class a referenceable(false){}; // prevent taking a reference to said type
class b referenceable(true){}; // same as class b {};
```

In terms of this proposal, the current default of the language is `referenceable(true)` .